CAB: Claude Code Architecture Guide

A Standardized Framework for Custom LLM Development

Version 0.8.1-draft

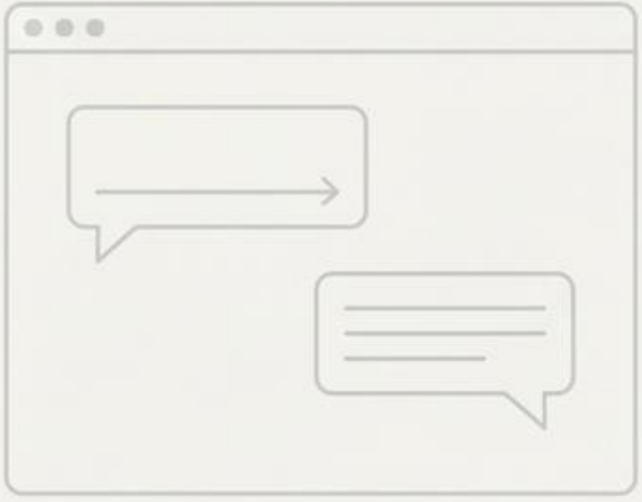
@daneyon/cc-architecture-builder By Daniel Kang (2026-03-05)

NotebookLM

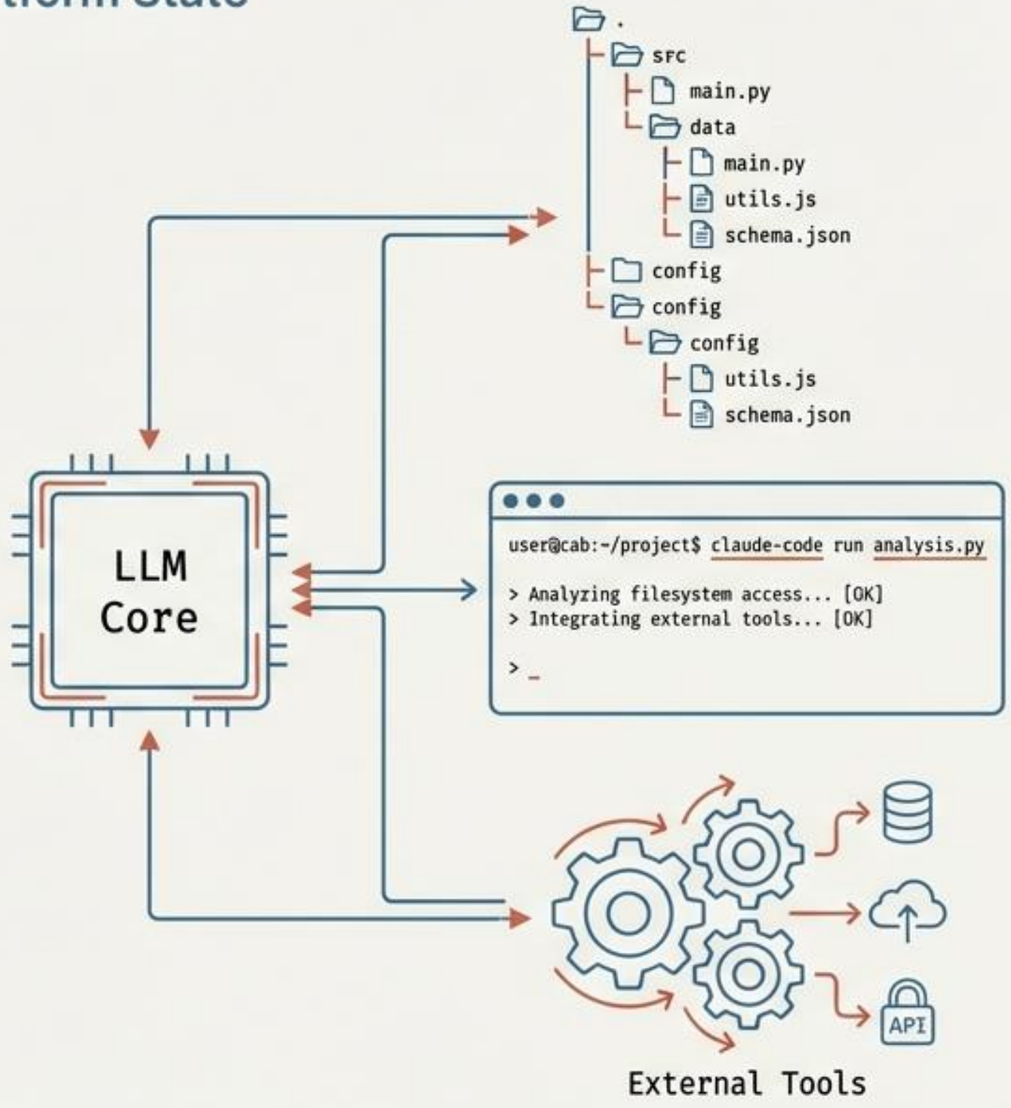
From Coding Assistant to Configurable AI Platform

Claude Code operates beyond standard chat interfaces. It is a highly configurable platform with deep filesystem access, tool integration, and extensible capabilities. CAB provides the standardized architecture to harness this platform for coding, research, enterprise automation, and domain-specific knowledge bases.

Legacy State



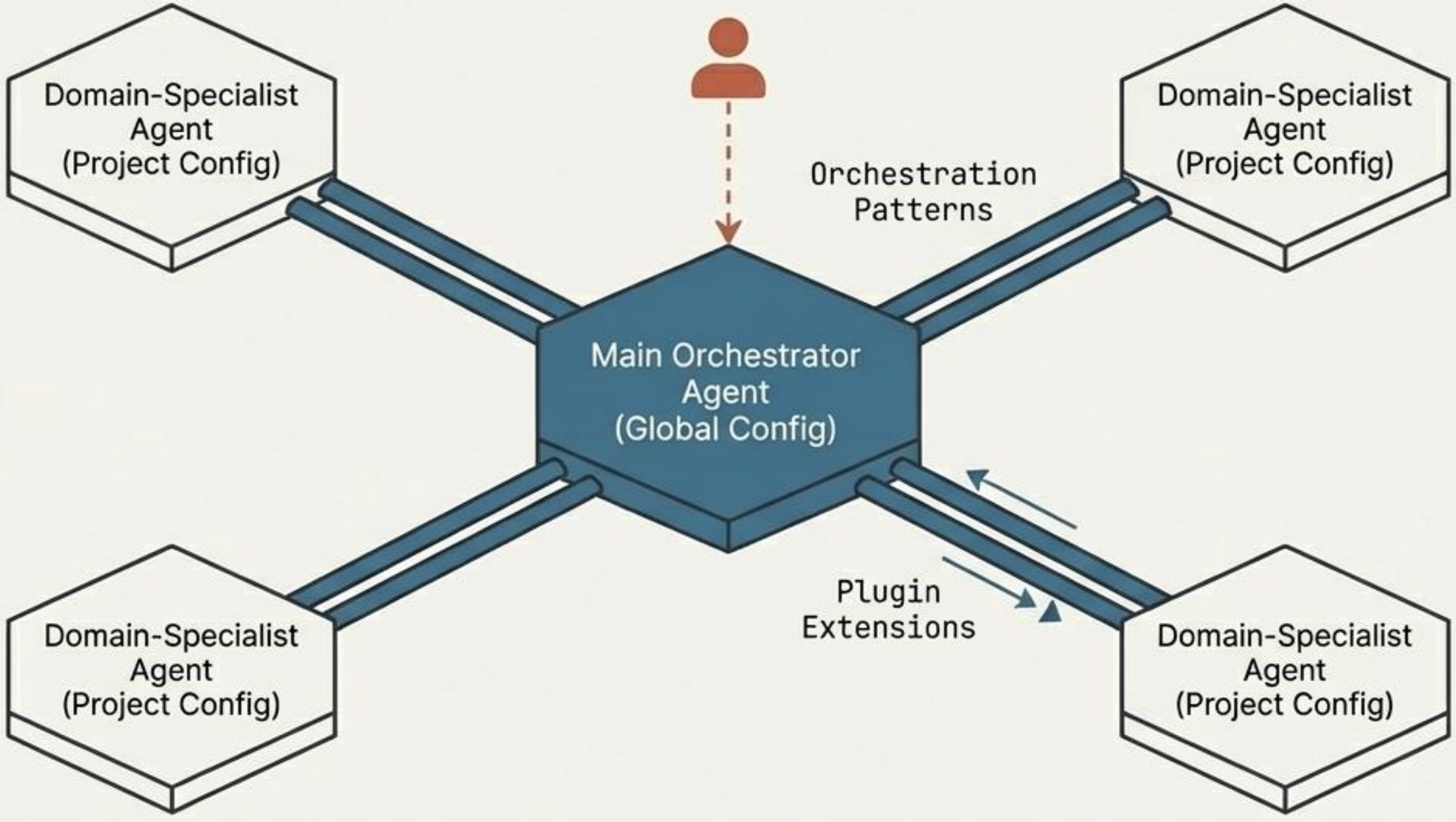
Platform State



NotebookLM

Building an Autonomous Multi-Agent Framework

The primary objective of CAB is to securely minimize human-in-the-loop dependencies. Humans are elevated to platform alignment and periodic knowledge base updates, while the framework handles the rest.



NotebookLM

8 Core Design Principles

Separation of Concerns

Global config stays personal; project config is distributable.

Progressive Disclosure

Load only what's needed; reference additional files via @imports.

Convention over Configuration

Follow official directory structures; minimize custom logic.

Git-Native Workflow

Version control everything; enable team collaboration.

Token Efficiency

Context is finite. Every token of config displaces productive output.

Security by Default

Private repos, credential exclusion, pre-publication review.

Verification as Requirement

Every agent, task, and gate requires a verification method.

Multi-Agent Autonomy

Design for independence; minimize human chokepoints.

NotebookLM

The Base Architecture: One Schema, Two Scopes

Both the global user configuration and the project plugin configuration share the exact same base schema. They differ only in physical location and scope.

Schema 1: Global

```
config.json
├── agents
│   └── orchestrator.json
├── patterns
│   └── domain_specialist.json
├── tools
│   └── workflow.json
└── settings
    ├── api_keys.json
    └── user_preferences.json
```

=

Schema 2: Plugin

```
config.json
├── agents
│   └── orchestrator.json
├── patterns
│   └── domain_specialist.json
├── tools
│   └── workflow.json
└── settings
    ├── api_keys.json
    └── user_preferences.json
```

Main Orchestrator + Synthesizer

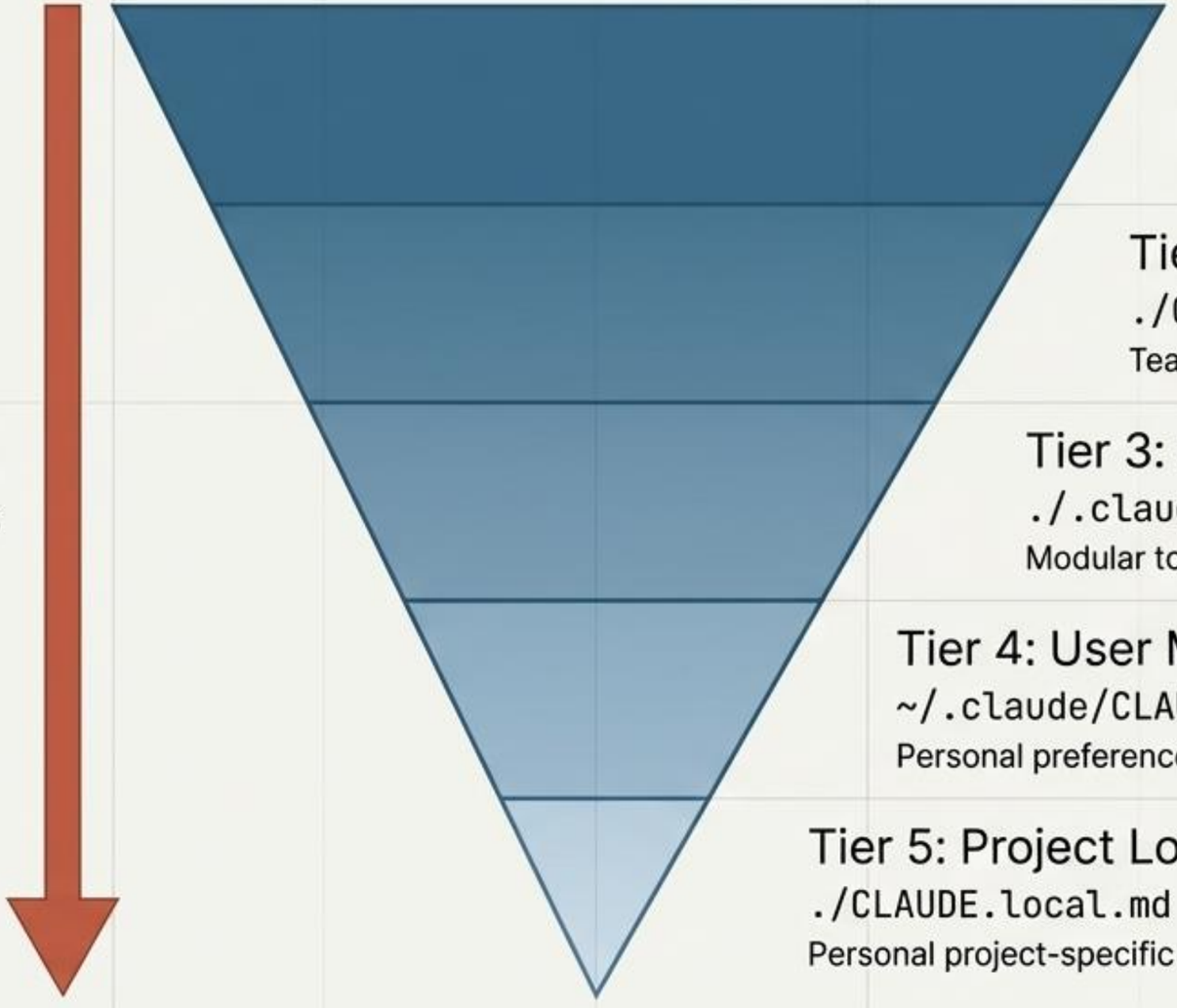
Location: `~/.claude/`
Tags: Private, Travels with you, Applies to ALL projects.

Domain-Specialist Agent

Location: `./project/`
Tags: Distributable, Team-shared, Domain-specific.

NotebookLM

The 5-Tier Memory Hierarchy



Precedence: Top tier loads first and takes precedence.

Tier 1: Enterprise Policy
`System paths`
Organization-wide standards | Shared with: All org users

Tier 2: Project Memory
`./CLAUDE.md`
Team-shared instructions | Shared with: Team via git

Tier 3: Project Rules
`./.claude/rules/*.md`
Modular topic-specific rules | Shared with: Team via git

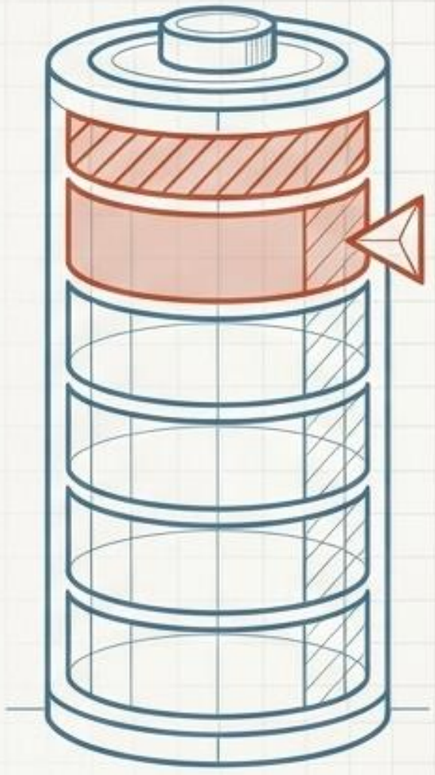
Tier 4: User Memory
`~/.claude/CLAUDE.md`
Personal preferences for all projects | Shared with: Just you

Tier 5: Project Local
`./CLAUDE.local.md`
Personal project-specific overrides | Shared with: Just you

NotebookLM

Invocation Patterns & Token Economics

Extension	Invocation	Trigger
Memory & Rules	Automatic	Loaded at session start
Skills	Model-invoked	Claude decides based on context
Subagents	Model or User	Auto-delegated or explicit
Commands	User-invoked	Explicitly typed (e.g., `/analyze`)
Hooks	Event-driven	Triggered by system events

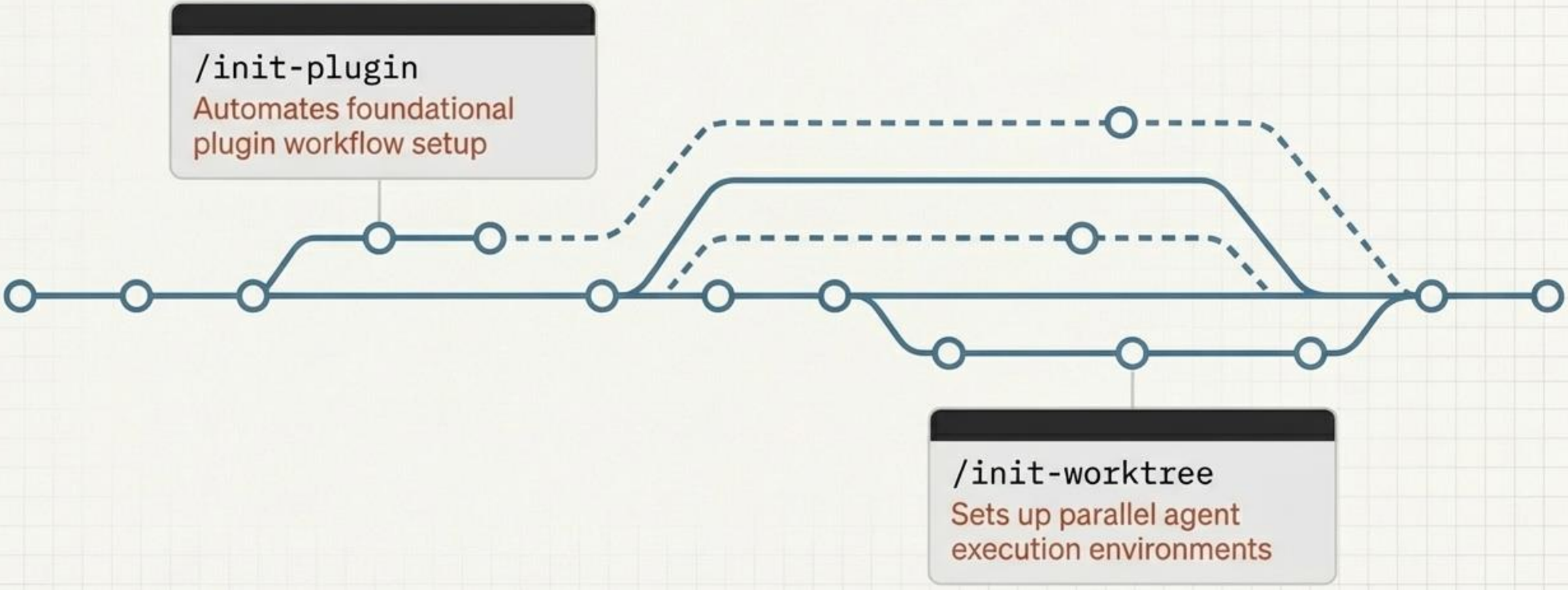


The context window is a shared, finite resource. Keep CLAUDE.md lean; use extensions for specialized knowledge.

NotebookLM

Git is Architecturally Required

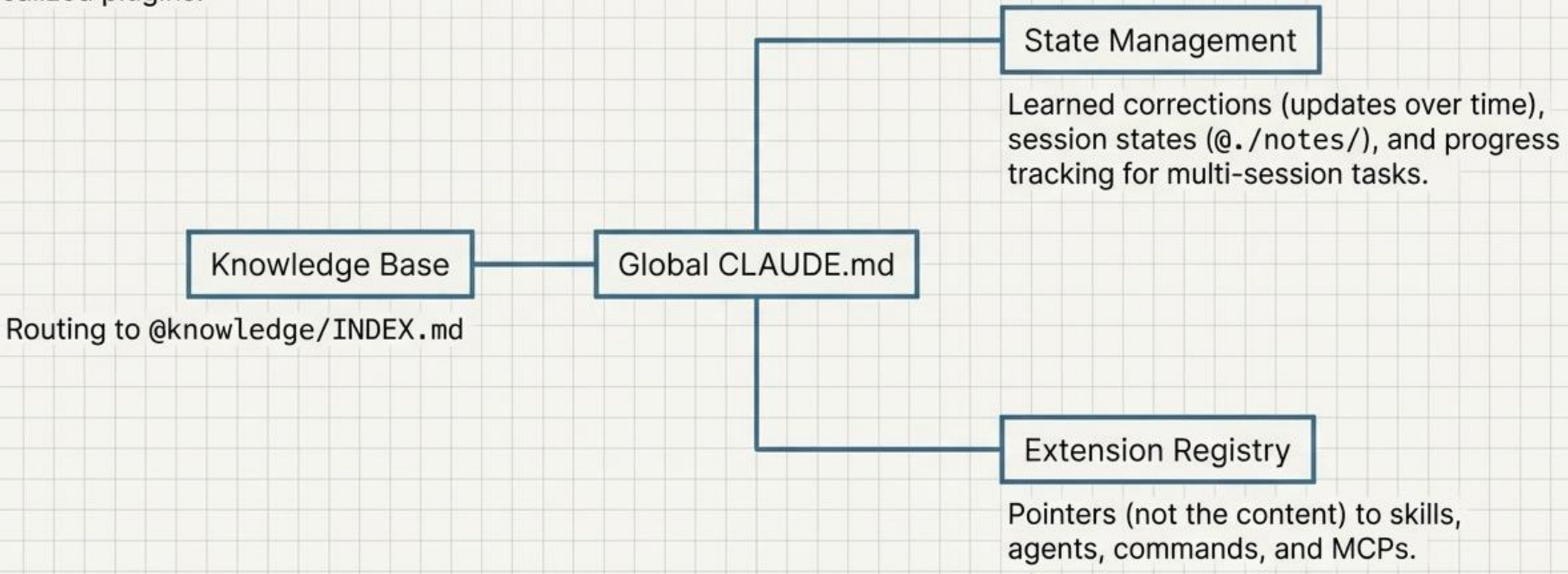
Version control isn't just a best practice; it is the engine for plugin distribution, collaboration, and parallel agent execution. Security defaults mandate private repositories and strict exclusion of .env, credentials.json, and .local files.



NotebookLM

Schema 1 Deep Dive: Global User Configuration

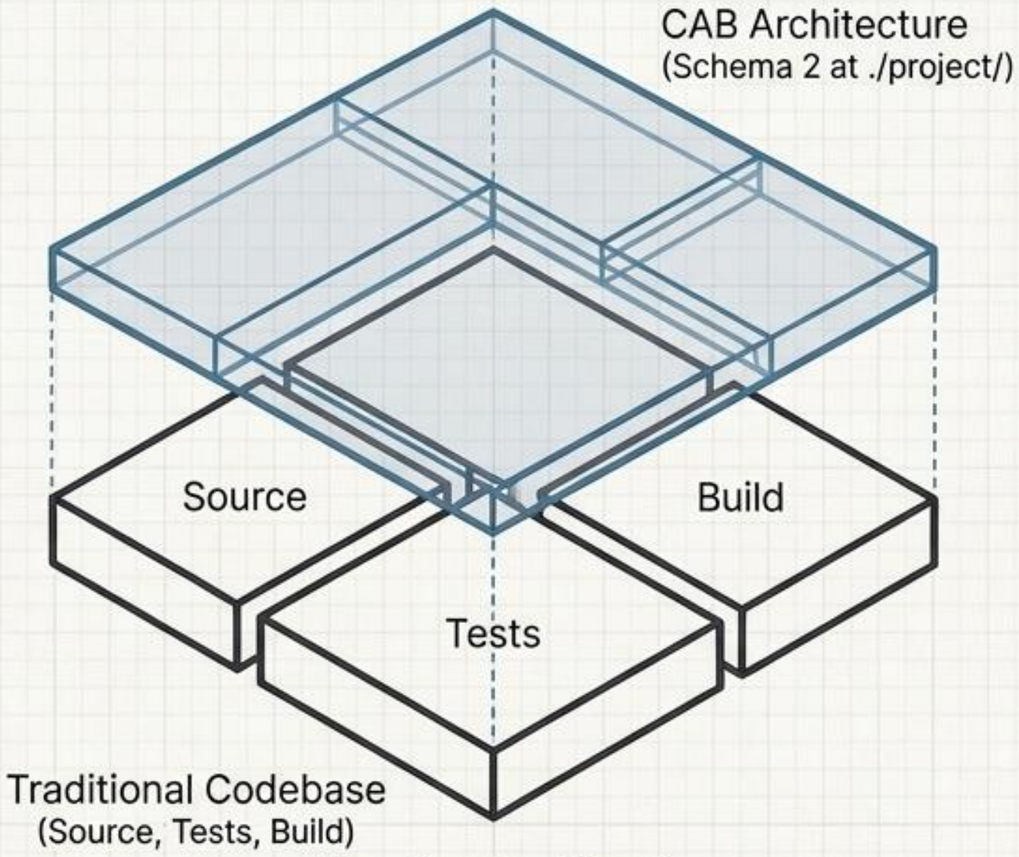
Located at `~/.claude/`, this is your personal baseline. Setting `agent: orchestrator` makes this layer the default synthesizer for all conversations, delegating to localized plugins.



NotebookLM

Schema 2 Deep Dive: The Distributable Plugin

Domain-specialist plugins are distributable packages that live right alongside existing code bases. The intelligence is injected without requiring architectural overhauls of your source code.



The Overlay Visual

NotebookLM

Anatomy of a Safe Execution Environment

Security Boundary

Metadata & Identity

plugin.json

```
{ ...
  "name": "My Plugin",
  "version": "1.0.0",
  ...
}
```

Defines the plugin for distribution.

settings.json

```
{
  "allowed_commands": [
    "build",
    "test"
  ],
  "deny_commands": [
    "deploy",
    "rm -rf"
  ]
}
```

Shared via git to pre-approve safe commands locally, keeping dangerous actions contained.

Domain Logic

Project CLAUDE.md

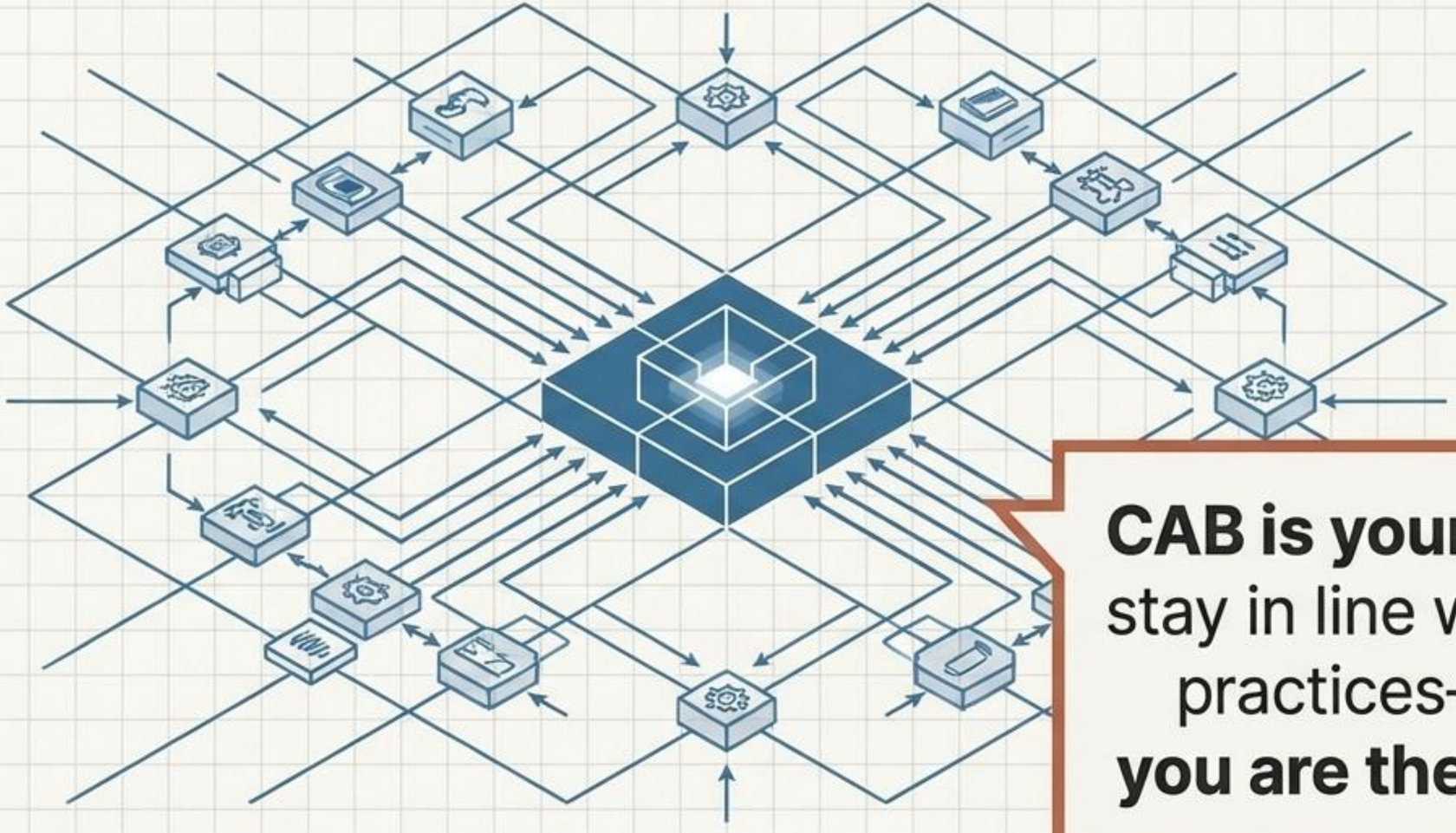
```
...
## Personal Customization:
@~/.claude/project-
preferences.md
...
```

Shows how users can personalize a shared plugin without blooming the shared repo.

NotebookLM

Driving the AI Platform

CAB provides the standardized architecture, but you provide the context engineering.
Standardized, secure, and extensible—your multi-agent framework is ready to scale.



CAB is your taxi to stay in line with best practices—and **you are the driver.**

NotebookLM